디지털논리회로설계 과제제2 결과보고서

소속	한양대학교 ERICA	과목명	디지털논리회로설계
학번	2023001421	이름	허은찬

1. 문제 1: 기본 논리식 VHDL 구현

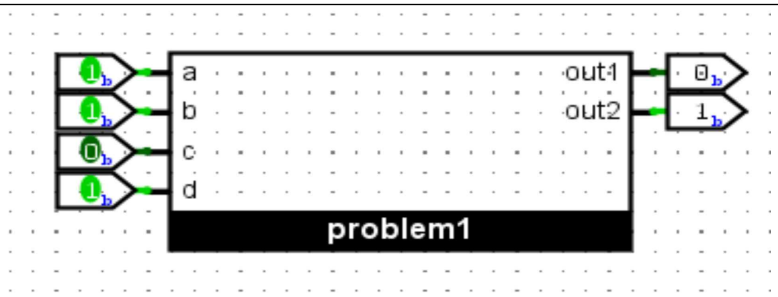
가. 설계 원리

4개의 1비트 입력 신호(a, b, c, d)를 받아 4-입력 AND 연산을 수행하는 out1(a and b and c and d)과 4-입력 OR 연산을 수행하는 out2(a or b or c or d)를 출력하도록 조합논리회로를 설계하였다.

나. VHDL 코드 및 회로 구성

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity problem1 is
5      port (
6          a    : in  std_logic;
7          b    : in  std_logic;
8          c    : in  std_logic;
9          d    : in  std_logic;
10         out1 : out std_logic;
11         out2 : out std_logic
12     );
13 end problem1;
14
15 architecture behavioral of problem1 is
16 begin
17     out1 <= a and b and c and d;
18     out2 <= a or b or c or d;
19 end behavioral;
```

<그림 1-1> 문제 1 (problem1) VHDL 작성 코드



<그림 1-2> 문제 1 (problem1) 회로 배선 및 출력 확인

다. 실험 결과

Logisim-evolution에서 VHDL Entity 블록을 생성하고 입출력 포트를 모두 1비트 std_logic으로 선언하였다. 입력 a, b, c, d에 모두 0을 인가했을 때 out1, out2 모두 0이 출력되었고, 입력 중 하나라도 1을 인가하면 OR 출력인 out2가 1로 바뀌었다. 4개 입력을 전부 1로 주었을 때만 AND 출력인 out1이 1로 나오는 것을 확인하였다.

2. 문제 2: 8-to-1 멀티플렉서 (Multiplexer)

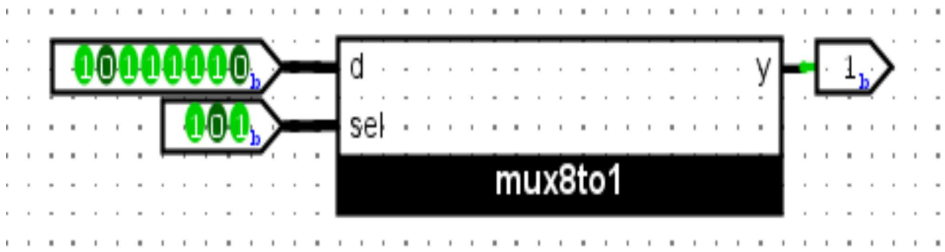
가. 설계 원리

8비트 데이터 입력 벡터 d(7 downto 0)와 3비트 선택 신호 sel(2 downto 0)을 입력받아, 선택 신호의 이진수 값(000~111)에 해당하는 비트 위치의 데이터를 단일 출력 y로 내보내는 8-to-1 멀티플렉서를 설계하였다.

나. VHDL 코드 및 회로 구성

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity mux8to1 is
5      port (
6          d   : in  std_logic_vector(7 downto 0);
7          sel : in  std_logic_vector(2 downto 0);
8          y   : out std_logic
9      );
10 end mux8to1;
11
12 architecture behavioral of mux8to1 is
13 begin
14     with sel select
15         y <= d(0) when "000",
16              d(1) when "001",
17              d(2) when "010",
18              d(3) when "011",
19              d(4) when "100",
20              d(5) when "101",
21              d(6) when "110",
22              d(7) when "111",
23              '0'  when others;
24 end behavioral;
```

<그림 2-1> 문제 2 (mux8to1) VHDL 작성 코드



<그림 2-2> 문제 2 (mux8to1) 회로 배선 및 선택 신호별 출력 확인

다. 실험 결과

VHDL 코드에서는 with ~ select 구문을 사용해 sel 값에 따라 d(0)부터 d(7)까지 매핑되도록 작성하였다. 회로도 상에서 입력 핀 d의 Data Bits를 8로, sel의 Data Bits를 3으로 설정하여

연결한 뒤 sel 값을 변경해 가며 해당 인덱스의 데이터 비트가 y로 정상 출력되는 것을 확인하였다.

3. 문제 3: 7-Segment 알파벳 출력 (a, b, c, d, f, g)

가. 설계 원리

3비트 선택 입력 sel에 따라 7-Segment Display에 알파벳 a, b, c, d, f, g 모양이 점등되도록 디코더 회로를 설계하였다. 대문자 B와 D는 숫자 8, 0과 구분이 안 되기 때문에 소문자 형태(b, d)로 표현하고 나머지 알파벳(A, C, F, G)은 대문자 형태로 세그먼트 비트 값을 지정하였다.

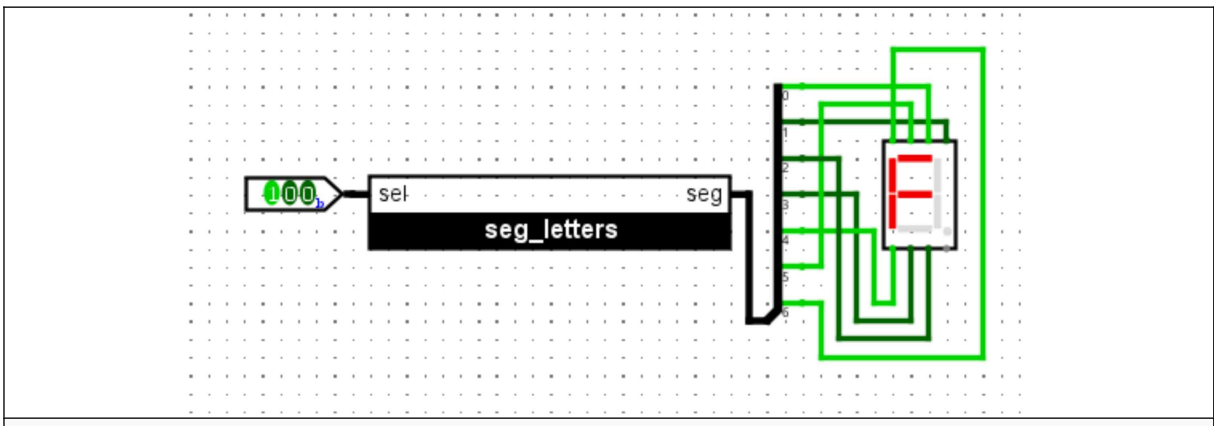
나. VHDL 코드 및 회로 구성

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity seg_letters is
    port (
        sel : in  std_logic_vector(2 downto 0);
        seg : out std_logic_vector(6 downto 0) -- gfedcba
    );
end seg_letters;

architecture behavioral of seg_letters is
begin
    process(sel)
    begin
        -- 비트 순서: "gfedcba" (1: 켜짐, 0: 꺼짐)
        case sel is
            when "000" => seg <= "1110111"; -- A
            when "001" => seg <= "1111100"; -- b
            when "010" => seg <= "0111001"; -- C
            when "011" => seg <= "1011110"; -- d
            when "100" => seg <= "1110001"; -- F
            when "101" => seg <= "0111101"; -- G
            when others => seg <= "0000000"; -- OFF
        end case;
    end process;
end behavioral;
```

<그림 3-1> 문제 3 (seg_letters) VHDL 작성 코드



<그림 3-2> 문제 3 (seg_letters) Splitter 분배 및 7-Segment 출력 확인

다. 실험 결과

Logisim의 7-Segment는 Active-High로 동작하므로 켜야 할 위치에 1을 출력하도록 case 문을 작성하였다. 7비트 출력 seg(6 downto 0)에 Splitter(Fan Out: 7, Bit Width In: 7)를 연결하여 0번부터 6번 가닥을 각각 세그먼트의 a~g 단자에 연결하였다. sel을 000부터 101까지 바꿀 때 A, b, C, d, F, G가 순서대로 출력되었고, 나머지 입력에서는 꺼지는 것을 확인하였다.

4. 문제 4: 25인 다수결 투표기 (25-person Voting Machine)

가. 설계 원리

25명의 투표 입력을 나타내는 25비트 벡터 votes(24 downto 0)에서 찬성('1')의 개수를 세어, 과반수인 13명 이상이 1일 때 출력 pass에 1을 내보내고 12명 이하일 때는 0을 출력하는 회로를 설계하였다.

나. VHDL 코드 및 회로 구성

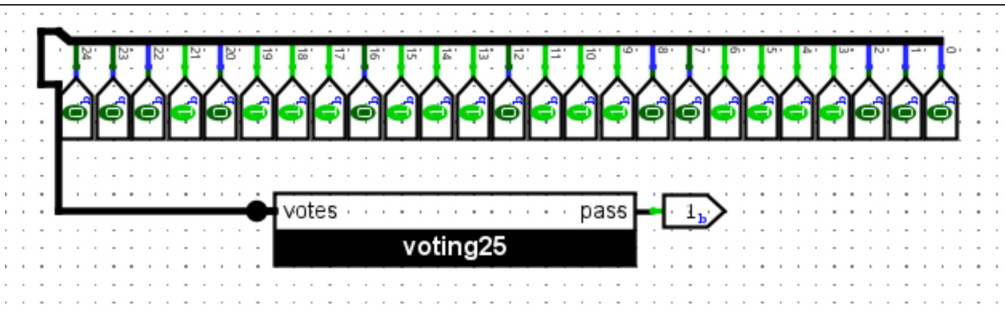
```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity voting25 is
    port (
        votes : in  std_logic_vector(24 downto 0);
        pass  : out std_logic
    );
end voting25;

architecture behavioral of voting25 is
begin
    process(votes)
        variable cnt : integer range 0 to 25;
    begin
        cnt := 0;
        for i in 0 to 24 loop
            if votes(i) = '1' then
                cnt := cnt + 1;
            end if;
        end loop;

        if cnt >= 13 then
            pass <= '1';
        else
            pass <= '0';
        end if;
    end process;
end behavioral;
```

<그림 4-1> 문제 4 (voting25) VHDL 작성 코드



<그림 4-2> 문제 4 (voting25) 25비트 입력 및 다수결 출력 확인

다. 실험 결과

process 문 내부에 정수형 변수 cnt를 선언하고 for 반복문(0 to 24)으로 1의 개수를 누적한 뒤, cnt >= 13 조건문으로 pass 출력을 결정하였다. 25비트 입력 핀에서 1의 개수가 12개일 때까지는 출력이 0이다가, 13개 이상 켜지는 순간 출력 핀이 1로 바뀌는 것을 확인하였다.

5. 문제 5: 13비트 1의 개수 카운터 및 7-Segment 출력 모듈

가. 설계 원리

13비트 입력 벡터 vt(12 downto 0) 안에 있는 '1'의 개수를 카운트하여 0~9개일 때는 숫자로, 10~15개일 때는 알파벳 A~F로 7-Segment에 표시하는 회로를 설계하였다. 또한 enable 신호(en)가 0일 때는 디스플레이를 끄고, 1일 때만 카운트 결과가 출력되도록 구성하였다.

나. VHDL 코드 및 회로 구성

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

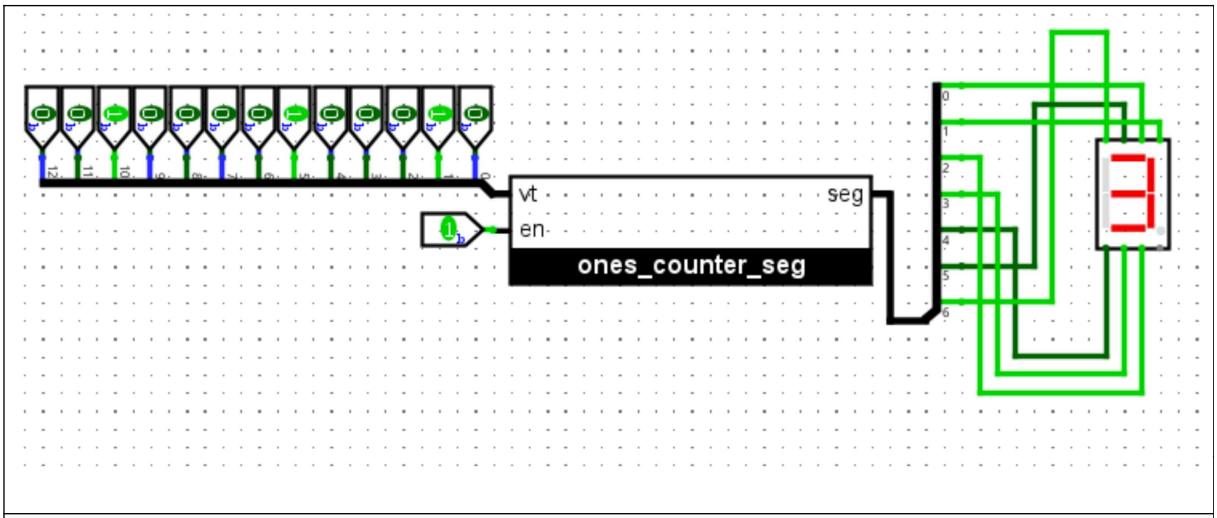
entity ones_counter_seg is
    port (
        vt  : in  std_logic_vector(12 downto 0);
        en  : in  std_logic;
        seg : out std_logic_vector(6 downto 0)
    );
end ones_counter_seg;

architecture behavioral of ones_counter_seg is
begin
    process(vt, en)
        variable cnt : integer range 0 to 15;
    begin
        cnt := 0;
        for i in 0 to 12 loop
            if vt(i) = '1' then
                cnt := cnt + 1;
            end if;
        end loop;

        if en = '0' then
            seg <= "0000000"; -- en='0'이면 화면 끔
```

```
27          else
28              case cnt is
29                  when 0  => seg <= "0111111"; -- 0
30                  when 1  => seg <= "0000110"; -- 1
31                  when 2  => seg <= "1011011"; -- 2
32                  when 3  => seg <= "1001111"; -- 3
33                  when 4  => seg <= "1100110"; -- 4
34                  when 5  => seg <= "1101101"; -- 5
35                  when 6  => seg <= "1111101"; -- 6
36                  when 7  => seg <= "0000111"; -- 7
37                  when 8  => seg <= "1111111"; -- 8
38                  when 9  => seg <= "1101111"; -- 9
39                  when 10 => seg <= "1110111"; -- A
40                  when 11 => seg <= "1111100"; -- b
41                  when 12 => seg <= "0111001"; -- C
42                  when 13 => seg <= "1011110"; -- d
43                  when 14 => seg <= "1111001"; -- E
44                  when 15 => seg <= "1110001"; -- F
45                  when others => seg <= "0000000";
46              end case;
47          end if;
48      end process;
49  end behavioral;
```

<그림 5-1> 문제 5 (ones_counter_seg) VHDL 작성 코드



<그림 5-2> 문제 5 (ones_counter_seg) 회로 배선 및 시뮬레이션 결과

다. 실험 결과

비트 조작을 편하게 하기 위해 1비트 입력 핀 13개를 Splitter로 묶어 vt 포트에 연결하였다. en이 0일 때는 입력 비트와 관계없이 7-Segment가 꺼져 있었고, en을 1로 바꾼 뒤 입력 비트를 하나씩 켜자 개수에 맞춰 숫자 0~9와 알파벳 A, b, C, d(최대 13개)가 정상적으로 표시되었다.

6. 고찰

이번 과제를 통해 기본 논리 게이트부터 멀티플렉서, 7-Segment 디코더, 그리고 for 반복문을 이용한 비트 카운터까지 VHDL로 작성하고 Logisim-evolution에서 직접 시뮬레이션해 보았다. 실습을 진행하면서 겪었던 몇 가지 오류와 해결 과정은 다음과 같다.

첫째, 회로도(.circ) 파일을 저장할 때 내부 VHDL 엔티티 이름(problem1)과 똑같은 이름으로 저장하려고 하니 이름 중복 에러가 발생하였다. Logisim에서는 내부 컴포넌트 이름과 파일 이름을 다르게 저장해야 한다는 것을 알게 되었다.

둘째, 처음 회로를 연결했을 때 출력에 파란색 U(Undefined)가 뜨거나 비트 폭 불일치(4/1) 경고가 나타났다. 이는 기본 생성 템플릿의 비트 폭이 남아있었기 때문이었으며, 코드 수정 후 에디터 상단의 검증(Validate) 버튼을 누르고 시뮬레이션 리셋(Ctrl + R)을 해주어야 변경된 코드가 회로에 정상 반영된다는 점을 확인하였다.

셋째, 3번과 5번 문제에서 Splitter를 사용할 때 Fan Out과 Bit Width In 설정 순서에 따라 여러 비트가 Output 0이나 Output 1 한쪽으로 몰리는 현상이 있었다. 속성 창에서 Bit 0부터 Bit 6까지 각각 Output 0~6에 1:1로 할당되도록 수정하여 해결하였다. 또한 5번 문제에서 13개의 입력 핀을 묶은 상단 Splitter의 왼쪽 끝 포트에 도선이 정확히 연결되지 않고 스쳐 지나가서 모든 비트를 1로 켜도 출력이 0으로만 나오던 문제가 있었는데, 연결 포트를 다시 맞춰 배선한 뒤 정상 동작을 확인할 수 있었다.